Project Euler: Problem 537

Version 1.0 (2015-12-5)

Contents

1 Problem 1

2 Basic solution 2

3 Parallel with polynomials 3

4 Example 4

5 Karatsuba multiplication 4

6 Fast Fourier Transform (FFT) 6

6.1 Introduction 6

6.2 Primitive m^{th} root of unity 6

6.3 Fast polynomial evaluation 7

6.4 Fast coefficient evaluation 8

6.5 Polynomial multiplication using FFT 9

6.6 Example 10

7 Summary of results found in the document 12

1 Problem

Let $\pi(x)$be the **prime counting function**, i.e. the number of prime numbers less of equal to x. For example, $\pi(1) = 0, \pi(2) = 1, \pi(100) = 25$.

Let $T(n, k)$ be the number of k-tuples $(x_1, \dots, x_k)$ which satisfy:

1. every x_i is a positive integer;
2. $\sum_{i=1}^{k} \pi(x_i) = n$

For example $T(3,3) = 19$.

The 19 tuples are (1,1,5), (1,5,1), (5,1,1), (1,1,6), (1,6,1), (6,1,1), (1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,1,2), (3,2,1), (1,2,4), (1,4,2), (2,1,4), (2,4,1), (4,1,2), (4,2,1), (2,2,2).

You are given $T(10,10) = 869\,985$ and $T(10^3, 10^3) \equiv 578\,270\,566 \pmod{1\,004\,535\,809}$.

Find $T(20\,000,20\,000) \bmod 1\,004\,535\,809$.

2 Basic solution

Split the k-tuple in two smaller tuples of length a and b, with $a + b = k$.

Let $s = \sum_{i=1}^{a} \pi(x_i)$, the sum of the prime counting function on the first tuple .

The sum on the second tuple is $n - s = \sum_{i=a+1}^{k} \pi(x_i)$.

Any tuple counted in $T(n,k)$ must be composed of a tuple counted in $T(s,a)$ and one counted in $T(n - s, b)$. Thus:

$$T(n,k) = \sum_{s=0}^{n} T(s,a)T(n-s,b) \tag{1}$$

If we define $T_{n,k} = [T(0,k), T(1,k), \dots, T(n,k)]$ as a vector of length $n+1$, then $T_{n,k}$ is the **convolution** of vectors $T_{n,a}$ and $T_{n,b}$ (written as $T_{n,k} = T_{n,a} * T_{n,b}$), truncated to length $n+1$. Each element of $T_{n,k}$ is defined by equation (1). We can thus express vector $T_{n,k}$ in terms of vectors $T_{n,a}$ and $T_{n,b}$ with $a, b < k$. We can continue splitting the tuples until we reach tuples of size one, i.e. $T_{n,1}$.

To ensure the validity of equation (1) for $a = 0$ or $b = 0$, we can also select $T_{n,0} = [1, 0, 0, \dots, 0]$, the **neutral element**, so that $T_{n,k} = T_{n,k} * T_{n,0} = T_{n,0} * T_{n,k}$.

Now consider the case $k = 1$. $T(n,1)$ is the count of positive integers x such that $\pi(x) = n$. So:

$$T(n,1) = \begin{cases} 1 & \text{if } n = 0 \\ p_{n+1} - p_n & \text{if } n > 0 \end{cases} \tag{2}$$

Where p_i is the ith prime.

$T(n,k)$ can be evaluated using **exponentiation by squaring** as shown in Algorithm 1. Its complexity is $O(n^2 \log k)$.

```
result ← T_{n,0}
squares ← T_{n,1}
while k ≠ 0
  if k is odd then
    result ← result * squares (truncated to length n + 1)
  squares ← squares * squares (truncated to length n + 1)
  k ← ⌊k/2⌋ (the brackets indicate the floor function)
return result_n.
```

Algorithm 1 Solution using convolution

The answer $T(n,k)$ is returned in the $(n + 1)^{\text{th}}$ element of the vector, i.e. $result_n$ using a zero based vector index.

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

3 Parallel with polynomials

Consider the following polynomial:

$$p_{n,k}(x) = T(0,k) + T(1,k)x + T(2,k)x^2 + \cdots + T(n,k)x^n = \sum_{j=0}^{n} T(j,k)x^j \tag{3}$$

The ordinary product between $p_{n,a}(x)$and $p_{n,b}(x)$ is given by:

$$p_{n,a}(x)p_{n,b}(x) = \sum_{j=0}^{a+b} c_j x^j \tag{4}$$

With:

$$c_j = \sum_{s=0}^{j} T(s,a)T(j-s,b) \text{ for } j \leq n. \tag{5}$$

Notice equation (5) is the same as equation (1), thus:

$$p_{n,a+b}(x) = p_{n,a}(x)p_{n,b}(x) \bmod x^{n+1} \tag{6}$$

$$p_{n,k}(x) = p_{n,1}^{k}(x) \bmod x^{n+1} \tag{7}$$

Where $(p(x) \bmod x^m)$ indicates the polynomial $p(x)$ truncated by removing all terms containing x raised to a power larger or equal to m, i.e. keeping only the powers of x smaller than m.

We can rewrite Algorithm 1 using polynomials as shown in Algorithm 2. Using long polynomial multiplication, its complexity is also $O(n^2 \log k)$.

```
result(x) ← 1
squares(x) ← p_{n,1}(x)
while k ≠ 0
  if k is odd then
    result(x) ← (result(x) squares(x)) mod x^{n+1}
  squares(x) ← squares(x)^2 mod x^{n+1}
  k ← ⌊k/2⌋ (the brackets indicate the floor function)
return the coefficient x^n from result.
```

Algorithm 2 Solution using polynomials

The answer $T(n,k)$ is returned in the coefficient of the term of degree n in $result$.

Note that truncating the polynomials to degree n after each multiplication is not necessary to obtain the correct answer, but is instrumental in creating an efficient algorithm. Computing $T(20000,20000)$ directly as $p_{20000,1}^{20000}(x)$ without truncation would generate a polynomial of degree 20000^2 = 400 000 000, big and slow.

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

4 Example

To illustrate the approach, we shall compute $T(3,3)$ using the polynomial view.

From equation (2) we have $T_{3,1} = [1,1,2,2]$.

From equation (3) we have $p_{3,1}(x) = 1 + x + 2x^2 + 2x^3$.

From equation (6) we have:

$$\begin{aligned} p_{3,2}(x) &= p_{3,1}^2(x) \bmod x^4 \\ &= (1 + x + 2x^2 + 2x^3)^2 \bmod x^4 \\ &= (1 + 2x + 5x^2 + 8x^3 + 8x^4 + 8x^5 + 4x^6) \bmod x^4 \\ &= 1 + 2x + 5x^2 + 8x^3 \end{aligned}$$

Applying equation (6) again we have:

$$\begin{aligned} p_{3,3}(x) &= p_{3,1}(x)p_{3,2}(x) \bmod x^4 \\ &= (1 + x + 2x^2 + 2x^3)(1 + 2x + 5x^2 + 8x^3) \bmod x^4 \\ &= (1 + 3x + 9x^2 + 19x^3 + 22x^4 + 26x^5 + 16x^6) \bmod x^4 \\ &= 1 + 3x + 9x^2 + 19x^3 \end{aligned}$$

$T(3,3)$ is the coefficient of the term in x^3, i.e. $T(3,3) = 19$.

5 Karatsuba multiplication

Polynomial multiplication, using long multiplication, has complexity $O(n^2)$, and when combined with exponentiation by squaring results in an algorithm with complexity $O(n^2 \log k)$.

However, by framing the algorithm using polynomial multiplication, we can leverage fast polynomial multiplication algorithms to create a faster algorithm. **Karatsuba multiplication** is one such algorithm.

Let $p(x)$ and $q(x)$ be polynomials of degree not exceeding $2m - 1$. We can write $p(x)$and $q(x)$ as:

$$p(x) = a(x) + b(x)x^m \tag{8}$$

$$q(x) = c(x) + d(x)x^m \tag{9}$$

Where $a(x)$, $b(x)$, $c(d)$ and $d(x)$ have degree not exceeding $m - 1$. Leading zeros are added if the polynomials do not have the same degree or the degree is an even number. The product becomes:

$$\begin{aligned} p(x)q(x) &= (a(x) + b(x)x^m)\,(c(x) + d(x)x^m) \\ &= a(x)c(x) + (a(x)d(x) + b(x)c(x))x^m + b(x)d(x)x^{2m} \\ &= a(x)c(x) + \big((a(x) + b(x))(c(x) + d(x)) - a(x)c(x) - b(x)d(x)\big)x^m + b(x)d(x)x^{2m} \end{aligned}$$

The last expression contains only three distinct products:

$$a(x)c(x),\ b(x)d(x) \text{ and } (a(x) + b(x))(c(x) + d(x)).$$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

The product of two polynomials of degree $2n-1$ is achieved using only three multiplications of polynomials of degree $n-1$, at the cost of a few extra additions and subtractions. However, since addition and subtraction have complexity $O(n)$, Karatsuba multiplication has asymptotic complexity $O(n^{\frac{\log 3}{\log 2}}) \approx O(n^{1.58})$ when used recursively on the smaller pieces, much faster than long multiplication for sufficiently large n.

Let's use Karatsuba multiplication to compute $p_{3,1}(x)p_{3,2}(x)$.

$$p_{3,1}(x) = (1+x) + (2+2x)x^2$$

$$p_{3,2}(x) = (1+2x) + (5+8x)x^2$$

Compute $a(x)+b(x)$:

$$(1+x)+(2+2x) = 3+3x$$

Compute $c(x)+d(x)$:

$$(1+2x)+(5+8x) = 6+10x$$

Compute $(a(x)+b(x))(c(x)+d(x))$:

$$(3+3x)(6+10x) = 18+48x+30x^2$$

Compute $a(x)c(x)$:

$$(1+x)(1+2x) = 1+3x+2x^2$$

Compute $b(x)d(x)$:

$$(2+2x)(5+8x) = 10+26x+16x^2$$

Compute $\big(a(x)+b(x)\big)\big(c(x)+d(x)\big) - a(x)c(x) - b(x)d(x)$:

$$(18+48x+30x^2)-(1+3x+2x^2)-(10+26x+16x^2) = 7+19x+12x^2$$

Compute $a(x)c(x) + \Big(\big(a(x)+b(x)\big)\big(c(x)+d(x)\big) - a(x)c(x) - b(x)d(x)\Big)x^2 + b(x)d(x)x^4$:

$$\begin{aligned}(1+3x+2x^2)+(7+19x+12x^2)x^2+(10+26x+16x^2)x^4 \\ = 1+3x+9x^2+19x^3+22x^4+26x^5+16x^6\end{aligned}$$

We obtain the same expression we obtained in the first example.

Using Karatsuba multiplication, the asymptotic complexity of our complete algorithm becomes $O(n^{\frac{\log 3}{\log 2}} \log k)$, which allows us to compute $T(50\,000, 50\,000) \equiv 587\,156\,969 \pmod{1\,004\,535\,809}$ in under a minute.

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

Toom-Cook multiplication generalizes Karatsuba multiplications by separating the polynomials in m pieces and using $2m-1$ multiplications. Karatsuba multiplication is equivalent to Toom-Cook with $m = 2$. Toom-Cook multiplication with m=3 (sometimes known as **Toom-3**) has asymptotic complexity $O(n^{\frac{\log 5}{\log 3}}) \approx O(n^{1.46})$ and can be used to compute $T(10^5, 10^5) \equiv 160\,727\,258 \pmod{1\,004\,535\,809}$ in under a minute. However we shall not provide a detailed explanation here.

6 Fast Fourier Transform (FFT)

6.1 Introduction

There is a unique polynomial of degree $m-1$ that passes through a given set of m distinct points. Thus the set $\{(x_0, r(x_0)),\ (x_1, r(x_1)),\ (x_2, r(x_2)), \dots, (x_{m-1}, r(x_{m-1}))\}$ completely defines polynomial $r(x)$ of degree $m-1$. If $r(x) = p(x)q(x)$ is the product of two polynomials, then the set can be written as:

$$\{(x_0, p(x_0)q(x_0)), (x_1, p(x_1)q(x_1)), (x_2, p(x_2)q(x_2)), \dots, (x_{m-1}, p(x_{m-1})q(x_{m-1}))\}$$

Notice that the multiplication at a given point is an ordinary multiplication, and thus all m products can be evaluated in $O(m)$ assuming we already evaluated $p(x)$ and $q(x)$ for the same values of x: $\{x_0, x_1, x_2, \dots, x_{m-1}\}$.

Normally, evaluating $p(x)$ at m distinct points would require $O(m^2)$ operations, so nothing would be gained over long polynomial multiplication. However, the FFT algorithm uses very specific values of x_i which allow evaluation of all $p(x_i)$ using $O(m \log m)$ operations, as well as finding the coefficients of $r(x) = p(x)q(x)$ once we know its value at m distinct points.

To keep things simple, we shall limit our discussion to the case where m is a power of two (i.e. $m = 2^t$). This condition can be met by selecting m to be the smallest power of two larger than the degree of the product we are computing, and extending the polynomials with terms having zeros as coefficients.

Note that engineers would call evaluating a polynomial given its coefficient as the **inverse discrete Fourier transform (IDFT)**, while computing the polynomial coefficients from values at specific point would be called the **discrete Fourier transform (DFT).** The **fast Fourier transform** is an algorithm used to evaluate the DFT/IDFT efficiently.

6.2 Primitive m^{th} root of unity

Let w be a **primitive m^{th} root of unity** modulo prime p, i.e. $w^m \equiv 1 \pmod{p}$ and $w^j \not\equiv 1 \pmod{p}$ for $0 < j < m$.

We shall select $x_i = w^i$ for $0 \le i < m$.

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

For a prime modulus p (e.g. 1 004 535 809) and m a power of two, $a^{p-1} \equiv 1 \pmod{p}$ and thus

$$w \equiv a^{\frac{p-1}{m}} \pmod{p} \tag{10}$$

is a primitive m^{th} root of unity if and only if

$$a^{\frac{p-1}{2}} \equiv w^{\frac{m}{2}} \equiv -1 \pmod{p}. \tag{11}$$

The easiest way to find a primitive m^{th} root of unity is simply to test different values of a until condition (11) is met.

For example, using $p = 1\,004\,535\,809$ and $m = 2^{21}$ (the largest power of two dividing $p - 1$), then $a \equiv 3 \pmod{p}$ is the smallest value that satisfies condition (11) which gives us $w \equiv 702\,606\,812 \pmod{p}$ from equation (10). Note that there are other values of w that we could use (other primitive m^{th} roots of unity). However, for our algorithm, any primitive m^{th} root of unity will do.

Note that if we want to compute the FFT over the **complex numbers** instead of the **finite field** of the integers modulo prime p, we would select $w = e^{\frac{2\pi i}{m}}$.

6.3 Fast polynomial evaluation

The first reason to select $x_i = w^i$ is to simplify the evaluation of polynomial $p(x)$.

From $p(x)$, construct two polynomials $p_{even}(x)$ and $p_{odd}(x)$ or degree $\frac{m}{2} - 1$ by selecting even and odd coefficients respectively:

$$p(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_{m-1} x^{m-1} \tag{12}$$

$$p_{even}(x) = p_0 + p_2 x + p_4 x^2 + \cdots + p_{m-2} x^{\frac{m}{2}-1} \tag{13}$$

$$p_{odd}(x) = p_1 + p_3 x + p_5 x^2 + \cdots + p_{m-1} x^{\frac{m}{2}-1} \tag{14}$$

$$p(x) = p_{even}(x^2) + x\, p_{odd}(x^2) \tag{15}$$

$$p(-x) = p_{even}(x^2) - x\, p_{odd}(x^2) \tag{16}$$

Equations (15) and (16) allow us to generate m values of $p(x)$ by computing $\frac{m}{2}$ values of $p_{even}(x)$ and $p_{odd}(x)$. Since $w^{\frac{m}{2}+i} = -w^i$, to evaluate $p(w^0), p(w^1), \ldots, p(w^{m-1})$, we need only to evaluate $p_{even}(w^2), p_{even}(w^4), \ldots, p_{even}(w^{m-2}), p_{odd}(w^2), p_{odd}(w^4), \ldots, p_{odd}(w^{m-2})$ and apply equations (15) and (16).

By applying this procedure recursively, we can evaluate $p(w^0), p(w^1), \ldots, p(w^{m-1})$ with complexity $O(m \log m)$.

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

```
Function FFT
Input: w, m, [p_0, p_1, ..., p_{m-1}]
Output: [p(w^0), p(w^1), ..., p(w^{m-1})]
If m = 1 return [p_0]
even ← FFT(w^2, m/2, [p_0, p_2, ..., p_{m-2}])
odd ← FFT(w^2, m/2, [p_1, p_3, ..., p_{m-1}])
result_i = { even_i + w^i odd_i          for 0 ≤ i < m/2
           { even_{i-m/2} + w^i odd_{i-m/2}  for m/2 ≤ i < m
return result.
```

Algorithm 3 FFT

6.4 Fast coefficient evaluation

The second reason for selecting $x_i = w^i$ for w a primitive m^{th} root of unity, is that it simplifies the evaluation of the coefficients of $r(x) = p(x)q(x)$.

Let:

$$r(x) = r_0 + r_1 x + r_2 x^2 + \cdots r_{m-1} x^{m-1}$$

$$F = [r(x_0), r(x_1), r(x_2), \dots, r(x_{m-1})]^T$$

$$U = \begin{bmatrix} w^0 & w^0 & w^0 & & w^0 \\ w^0 & w^1 & w^2 & \cdots & w^{m-1} \\ w^0 & w^2 & w^4 & & w^{2(m-1)} \\ & \vdots & & \ddots & \vdots \\ w^0 & w^{m-1} & w^{2(m-1)} & \cdots & w^{(m-1)(m-1)} \end{bmatrix}$$

$$R = [r_0, r_1, r_2, \dots, r_{m-1}]^T$$

Then:

$$F = UR \tag{17}$$

If we want to find the coefficients of $r(x)$, we can do so by:

$$R = U^{-1}F \tag{18}$$

Let's consider the matrix V we obtain by substituting $w \leftarrow w^{-1}$ in U:

$$V = \begin{bmatrix} w^0 & w^0 & w^0 & & w^0 \\ w^0 & w^{-1} & w^{-2} & \cdots & w^{-(m-1)} \\ w^0 & w^{-2} & w^{-4} & & w^{-2(m-1)} \\ & \vdots & & \ddots & \vdots \\ w^0 & w^{-(m-1)} & w^{-2(m-1)} & \cdots & w^{-(m-1)(m-1)} \end{bmatrix}$$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

Let $Z = U.V$.

Z has element $z_{i,j}$ at row i, column j:

$$z_{i,j} = \sum_{k=0}^{m-1} u_{i,k} v_{k,j} = \sum_{k=0}^{m-1} w^{ik} w^{-kj} = \sum_{k=0}^{m-1} w^{(i-j)k} \tag{19}$$

When $i = j, w^{(i-j)k} = w^0 = 1$ and thus $z_{i,i} = \sum_{k=0}^{m-1} 1 = m$.

When $i \neq j$, let $g = \mathrm{GCD}(i-j, m)$ equation (19) becomes:

$$z_{i,j} = \sum_{l=0}^{g-1} \sum_{k=0}^{\frac{m}{g}-1} w^{(i-j)(\frac{m}{g}l+k)} \tag{20}$$

$$z_{i,j} = \sum_{l=0}^{g-1} \sum_{k=0}^{\frac{m}{2g}-1} \left(w^{(i-j)(\frac{m}{g}l+k)} + w^{(i-j)(\frac{m}{g}l+k+\frac{m}{2g})} \right) \tag{21}$$

$$z_{i,j} = \sum_{l=0}^{g-1} \sum_{k=0}^{\frac{m}{2g}-1} w^{(i-j)(\frac{m}{g}l+k)} \left(1 + \left(w^{\frac{m}{2}} \right)^{\frac{i-j}{g}} \right) \tag{22}$$

Since m is a power of 2 greater than $i-j$, $\frac{i-j}{g}$ is odd. Together with $w^{\frac{m}{2}} \equiv -1 \pmod{p}$, we obtain:

$$z_{i,j} = \sum_{l=0}^{g-1} \sum_{k=0}^{\frac{m}{2g}-1} w^{(i-j)(\frac{m}{g}l+k)} (1-1) = 0 \tag{23}$$

Thus:

$$z_{i,j} = \begin{cases} m & if\ i = j \\ 0 & if\ i \neq j \end{cases} \tag{24}$$

So the matrix product $U.V$ is equal to m times the identity matrix:

$$U.V = mI \tag{25}$$

$$U^{-1} = m^{-1} V \tag{26}$$

Equation (26) tells us that in order to compute the coefficients of $r(x)$, all we need is to compute $\frac{1}{m} FFT(w^{-1}, [r(x_0), r(x_1), r(x_2), \dots, r(x_{m-1})])$.

6.5 Polynomial multiplication using FFT

We can combine the various results we have seen so far to create a fast polynomial multiplication algorithm based on FFT. Let:

$$p(x) = p_0 + p_1 x + p_2 x^2 + \cdots p_n x^n$$

$$q(x) = q_0 + q_1 x + q_2 x^2 + \cdots q_n x^n$$

$$r(x) = p(x)q(x) = r_0 + r_1 x + r_2 x^2 + \cdots r_{2n} x^{2n}$$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

```
Function MultiplyFFT
Input: p, [p_0, p_1, ..., p_n], [q_0, q_1, ..., q_n]
Output: [r_0, r_1, ..., r_2n]
m ← 2^⌈log_2(2n+1)⌉ where the brackets indicate the ceiling function
If m ∤ p − 1 return error.
a ← 2
While a^((p−1)/2) ≢ −1 (mod p)
   a ← a + 1
w ← a^((p−1)/m) mod p
x ← FFT(w, m, [p_0, p_1, ..., p_{m−1}]) vector padded to length m with zeros
y ← FFT(w, m, [q_0, q_1, ..., q_{m−1}]) vector padded to length m with zeros
Return m^{−1} FFT(w^{−1}, m, [x_0 y_0, x_1 y_1, x_2 y_2, ..., x_{m−1} y_{m−1}])
```

Algorithm 4 Multiplication using FFT

The time required to execute this algorithm is dominated by the three FFT invocations, so the asymptotic complexity is the same as the FFT, i.e. $O(m \log m)$. Solving the complete problem, including the exponentiation by squaring, has asymptotic complexity $O(m \log m \log k)$, which allows us to compute $T(10^6, 10^6) \equiv 10\ 833\ 650 \pmod{1\ 004\ 535\ 809}$ in under a minute.

Algorithm 4 requires that $m|p-1$. It is the simplest form of FFT. More advanced forms can cope with cases where $p-1$ does not contain a large power of two, at the cost of extra complexity. In the current problem, p was selected so that $2^{21}|p-1$, making FFT computations simpler.

6.6 Example

Let's use FFT multiplication to compute $p_{3,1}(x)p_{3,2}(x) \bmod 97$. We selected $p = 97$ to keep numbers small.

1. $n = 3$
2. $m = 8$
3. $a = 5$
4. $w \equiv a^{\frac{p-1}{m}} \equiv 11^{63} \equiv 64 \pmod{97}$
5. $w^{-1} \equiv 47 \pmod{97}$
6. $m^{-1} \equiv 85 \pmod{97}$
7. $\mathrm{x} = \mathrm{FFT}(64,8,[1,1,2,2,0,0,0,0])$
   - 7.1. $\mathrm{even} <- \mathrm{FFT}(22,4,[1,2,0,0])$
     - 7.1.1. $\mathrm{even} <- \mathrm{FFT}(96,2,[1,0])$
       - 7.1.1.1. $\mathrm{even} <- \mathrm{FFT}(1,1,[1]) = [1]$
       - 7.1.1.2. $\mathrm{odd} <- \mathrm{FFT}(1,1,[0]) = [0]$
       - 7.1.1.3. $\mathrm{result} <- [1,1]$
     - 7.1.2. $\mathrm{odd} <- \mathrm{FFT}(96,2,[2,0])$
       - 7.1.2.1. $\mathrm{even} <- \mathrm{FFT}(1,1,[2]) = [2]$
       - 7.1.2.2. $\mathrm{odd} <- \mathrm{FFT}(1,1,[0]) = [0]$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte

7.1.2.3. result $<-[2,2]$
7.1.3. result $<-[3,45,96,54]$
7.2. odd $<-\mathrm{FFT}(22,4,[1,2,0,0])$
7.2.1. even $<-\mathrm{FFT}(96,2,[1,0])$
7.2.1.1. even $<-\mathrm{FFT}(1,1,[1]) = [1]$
7.2.1.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
7.2.1.3. result $<-[1,1]$
7.2.2. odd $<-\mathrm{FFT}(96,2,[2,0])$
7.2.2.1. even $<-\mathrm{FFT}(1,1,[2]) = [2]$
7.2.2.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
7.2.2.3. result $<-[2,2]$
7.2.3. result $<-[3,45,96,54]$
7.3. result $<-[6,15,74,38,0,75,21,70]$
8. y $= \mathrm{FFT}(64,8,[1,2,5,8,0,0,0,0])$
8.1. even $<-\mathrm{FFT}(22,4,[1,5,0,0])$
8.1.1. even $<-\mathrm{FFT}(96,2,[1,0])$
8.1.1.1. even $<-\mathrm{FFT}(1,1,[1]) = [1]$
8.1.1.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
8.1.1.3. result $<-[1,1]$
8.1.2. odd $<-\mathrm{FFT}(96,2,[5,0])$
8.1.2.1. even $<-\mathrm{FFT}(1,1,[5]) = [5]$
8.1.2.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
8.1.2.3. result $<-[5,5]$
8.1.3. result $<-[6,14,93,85]$
8.2. odd $<-\mathrm{FFT}(22,4,[2,8,0,0])$
8.2.1. even $<-\mathrm{FFT}(96,2,[2,0])$
8.2.1.1. even $<-\mathrm{FFT}(1,1,[2]) = [2]$
8.2.1.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
8.2.1.3. result $<-[2,2]$
8.2.2. odd $<-\mathrm{FFT}(96,2,[8,0])$
8.2.2.1. even $<-\mathrm{FFT}(1,1,[8]) = [8]$
8.2.2.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
8.2.2.3. result $<-[8,8]$
8.2.3. result $<-[10,81,91,20]$
8.3. result $<-[16,57,58,18,93,68,31,55]$
9. Multiply x and y element by element: $[96,79,24,5,0,56,69,67]$
10. z $= \mathrm{FFT}(47,8,[96,79,24,5,0,56,69,67])$
10.1. even $<-\mathrm{FFT}(75,4,[96,24,0,69])$
10.1.1. even $<-\mathrm{FFT}(96,2,[96,0])$
10.1.1.1. even $<-\mathrm{FFT}(1,1,[96]) = [96]$
10.1.1.2. odd $<-\mathrm{FFT}(1,1,[0]) = [0]$
10.1.1.3. result $<-[96,96]$
10.1.2. odd $<-\mathrm{FFT}(96,2,[24,69])$
10.1.2.1. even $<-\mathrm{FFT}(1,1,[24]) = [24]$
10.1.2.2. odd $<-\mathrm{FFT}(1,1,[69]) = [69]$
10.1.2.3. result $<-[93,52]$
10.1.3. result $<-[92,19,3,76]$
10.2. odd $<-\mathrm{FFT}(75,4,[79,5,56,67])$
10.2.1. even $<-\mathrm{FFT}(96,2,[79,56])$
10.2.1.1. even $<-\mathrm{FFT}(1,1,[79]) = [79]$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.

10.2.1.2. $\text{odd} <- \text{FFT}(1,1,[56]) = [56]$
10.2.1.3. $\text{result} <- [38, 23]$
10.2.2. $\text{odd} <- \text{FFT}(96,2,[5, 67])$
10.2.2.1. $\text{even} <- \text{FFT}(1,1,[5]) = [5]$
10.2.2.2. $\text{odd} <- \text{FFT}(1,1,[67]) = [67]$
10.2.2.3. $\text{result} <- [72, 35]$
10.2.3. $\text{result} <- [13, 29, 63, 17]$
10.3. $\text{result} <- [8, 24, 72, 55, 79, 14, 31, 0]$
11. Multiply by $m^{-1} \equiv 85 \pmod{p}$: [1, 3, 9, 19, 22, 26, 16, 0]

The result at the last step corresponds to the polynomial:

$$1 + 3x + 9x^2 + 19x^3 + 22x^4 + 36x^5 + 16x^6$$

$T(3,3)$ is the coefficient of the term in x^3, i.e. $T(3,3) \equiv 19 \pmod{97}$.

7 Summary of results found in the document

$T(50\,000, 50\,000) \equiv 587\,156\,969 \pmod{1\,004\,535\,809}$
$T(10^5, 10^5) \equiv 160\,727\,258 \pmod{1\,004\,535\,809}$
$T(10^6, 10^6) \equiv 10\,833\,650 \pmod{1\,004\,535\,809}$

Copyright Project Euler, further distribution without the consent of the author(s) prohibited.
Author: Martin Piotte